Project Proposal: Smartphone Gait-Based Owner Verification

Efe Ozbal - S5996821
Sigurdur Haukur Birgisson - S5982960
Tijje Steeman - S5333644
Will Meeus - S5863708

May 1, 2026

1 Preliminary Domain Knowledge

During our preliminary research, we were especially interested in machine learning projects that make use of data already collected by everyday devices. Smartphones stood out as a strong example because they continuously record information from built-in sensors such as accelerometers and gyroscopes. This suggests that phones may support useful security or health-related applications without requiring extra hardware.

For our project, we want to apply this idea to a practical security problem: determining whether a phone remains under its owner's control while it is being carried by someone who is walking. This matters because theft or unauthorized access does not always happen in an obvious way. A phone might be taken and walked away with, or it might be picked up briefly for a nearby action such as making a payment and then returned. In either case, identifying only the exact moment the phone is taken is usually not enough.

Our main idea is that a person's gait may act as a behavioral fingerprint. A phone carried while walking records repeated motion patterns, and we believe these signals may contain user-specific information such as stride timing, acceleration structure, and frequency patterns. We found a previous study and repository that approached a related problem as a multiclass classification task, which suggests that different people may produce distinguishable motion signatures with high predictive accuracy.

Based on this, our project aims to model walking sequences from smartphone accelerometer and gyroscope data and map them into an embedding space. We then plan to use cosine similarity or another comparison method to estimate whether a new walking segment is likely to come from the phone's owner or from someone else. In other words,

the machine learning task is open-set owner-versus-non-owner verification based on gait-related sensor data collected during walking.

2 Preliminary Data Exploration

The dataset we found on GitHub contains 3-axis gyroscope readings (GYR) and 3-axis accelerometer readings (ACC) over time, where Figure 1 shows the sensor axes. Figure 2a shows an example signal. This gives us the channels ACC_x, ACC_y, ACC_z, GYR_x, GYR_y, and GYR_z, each sampled at 50 Hz. As an attempt to filter out the high-frequency noise, we applied a low-pass filter to the raw signals, as shown in Figure 2b. Both raw and smoothed signals show a periodic pattern that is expected from the previous domain knowledge of the walking motion. Hence, we will extract features from frequency domain. Since we frame the task as open-set verification, class balance during training can be controlled by balancing genuine and impostor comparisons within each fold.

Other possible features we consider are the absolute acceleration, which is computed as $\sqrt{ACC_x^2 + ACC_y^2 + ACC_z^2}$, and absolute angular velocity, which is computed as $\sqrt{GYR_x^2 + GYR_y^2 + GYR_z^2}$. These features are expected to be informative because they capture the overall magnitude of acceleration and angular velocity regardless of direction, which can be indicative of the intensity and dynamics of the walking motion.

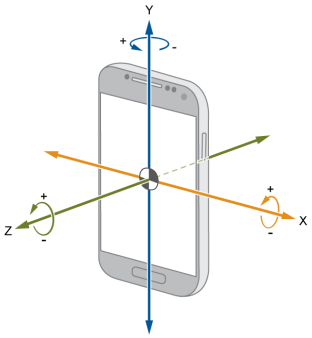
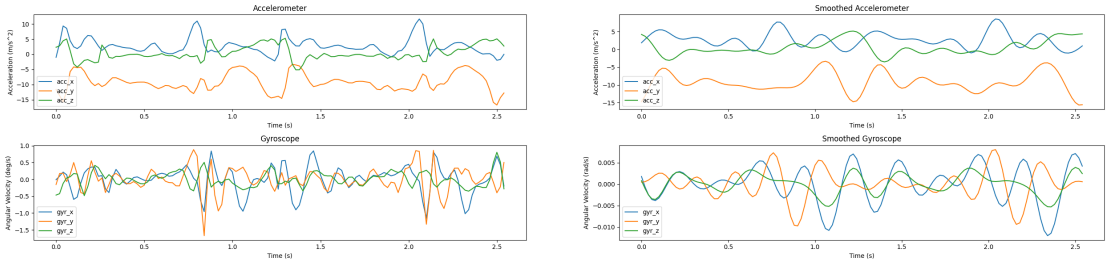


Figure 1: Sensor axes for accelerometer and gyroscope readings [1]



(a) Raw Sensor Signals Over Time

(b) Smoothed Sensor Signals Over Time

Figure 2: Time-series sensor signal examples

Since the task we are framing is a one-vs-rest verification problem, the class balance can be controlled during training by pairing each positive example with a negative comparison example.

We also computed the correlation matrix of the six sensor channels, which is shown in Figure 3. Even though majority of absolute value of channel correlations are close to zero (¡ 0.1), there can be seen a slight positive correlation of 0.355 between GYR_x and ACC_z. They are predicted to be correlated because both channels are influenced by the inward motion of the leg during stepping forward, where GYR_x captures the femur rotation and ACC_z captures the vertical acceleration from leg lifting.

Accompanying it, there is a negative correlation of -0.229 between GYR_z and ACC_x, which can be explained by the swinging motion of the leg during walking. Lastly, the correlation of 0.153 between ACC_z and ACC_y is believed to be, again, due to the inward motion of the leg during stepping forward. It should be noted that these correlations are weak and could be due to random chance.

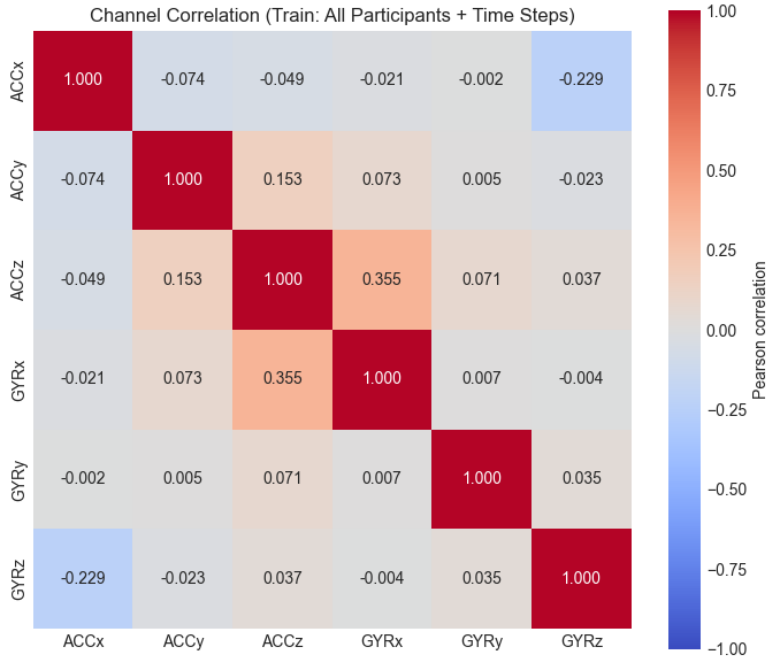


Figure 3: Correlation Matrix of Sensor Readings

As a dimensionality reduction attempt, we applied t-SNE, with 118 classes (one for each embedding in our dataset), to the six sensor channels, which is shown in Figure 4. First thing to notice here is that the peripheral clusters are well separated from each other, indicating easier classification of those samples. However, the central cluster is mixed and dense, which is expected for a non-linear model to separate these classes.

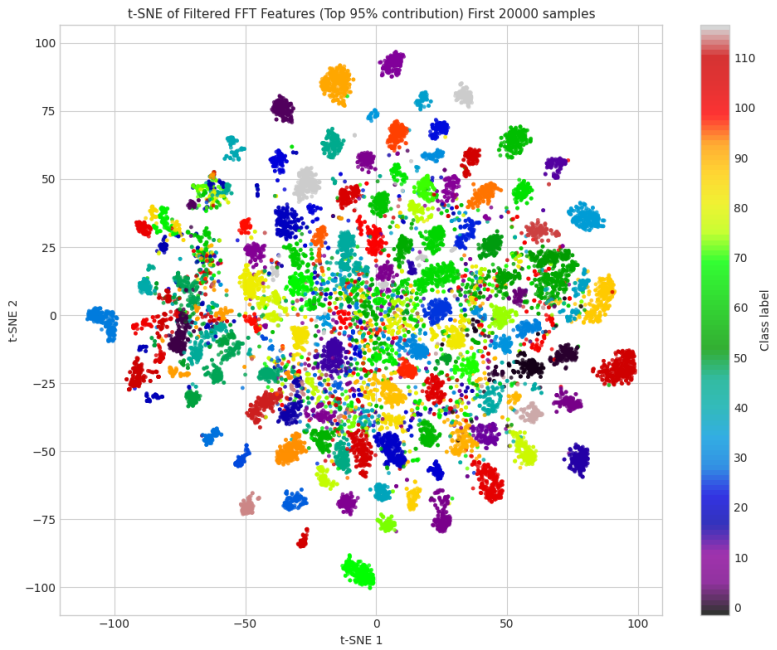


Figure 4: t-SNE plot of the dataset

There is no missing data or outliers in the dataset.

3 Proposed Preprocessing

We would like to propose a preprocessing pipeline that is organized into two stages. First, we will apply a shared pipeline for all experiments, starting with data quality checks. Since we have confirmed that there are no missing values, we will look for any outliers and sensor artifacts (flatline sections or implausible accelerometer/gyroscope spikes) to ensure the cleanliness of the data. Segmentation has already been performed on the raw data, and each sample will be represented as a segmented window. Since the dataset provides both the raw data and the processed, segmented data, we explain the motivation for using their segmentation and what has already been done. The original paper's segmentation strategy uses cumulative acceleration to identify each two-step window, which are of variable length. To create a fixed-length representation, they apply linear interpolation to produce windows of length 128, so the network always receives a constant temporal input size regardless of walking speed. Many papers, including the original, have found that meaningful segmentations improve predictive accuracy, so we will use this baseline data.

In a different approach than the original paper, we will use strict subject-level splitting

to avoid identity leakage. We think this is the safest approach, with clean separation maintained throughout by participant. We will use a 90/10 data split for train/test, with the training set used to construct $k \sim 5$ folds by subject groups with 80/20 for training and validation, which gives us good averaging across validation folds. The full training set will be used later. This depends on the training intensity we find; we may adjust the k value accordingly. We will use per-channel z-score normalization computed only on the training partition and then reused for validation and test data. We will have to precompute the folds since the z-score will be specific for the validation run, then the test set z-score computed on the full training set statistics.

After the shared stage, we branch by model type. For the baseline model, we will prepare manually computed time-domain or frequency-domain features per window (such as mean, standard deviation, turning points, and range, or FFT cumulative energy contribution), and then perform model-specific feature selection during baseline testing. We have already experimented with FFT features ranked by importance to prediction with success. For the main deep-learning track, we will use the segmented 128-length windows directly, and also test whether feature selection benefits the deeper model. As an important distinction, we have chosen not to construct triplet pairs during preprocessing. This is motivated by wanting to test different sampling strategies, such as random selection, hard negative mining, and batch-level negatives, and to decide later based on efficiency.

4 Proposed Model(s) and Baseline

For the baseline, we will use a nearest-centroid or prototype-based classifier on the FFT features from preprocessing. This gives us a simple and cheap reference point that is easy to interpret and useful as a comparison against the transformer model.

For the main model, we plan to use a transformer because it can learn non-linear relationships across the time dimension of the gait windows instead of relying only on hand-crafted summary features. The transformer will be trained with triplet loss as the primary training objective, so that windows from the same person move closer together in embedding space and windows from different people move farther apart. In addition, we will explore other metric-learning optimization techniques as extensions, while keeping triplet loss as the main loss function.

We will use dropout, L2 regularization, and normalization layers to reduce overfitting. Early stopping will be based on the validation equal error rate (EER), since that is the clearest summary of verification performance for this task. These choices should also make the model easier to compress later, because a less overfit model is usually less sensitive when weights are pruned or quantized.

For deployment, we want two levels of success, one for the base grade and one extra. The baseline goal is to expose the model through an API endpoint so the project is functional and can return predictions from sensor input. The extra goal is on-device deployment, so the phone itself can run the model locally with as little dependency on a server as possible. This would be the more applied version of the project and is the main stretch goal.

5 Proposed Evaluation

For the main open-set verification task, we will evaluate the model primarily with false acceptance rate (FAR), false rejection rate (FRR), and equal error rate (EER), because these metrics directly reflect how well the system accepts genuine users and rejects impostors. We may still report accuracy, precision, recall, F1 score, confusion matrices, and AUROC as secondary metrics when they help with comparison, but these are less central to the verification setting than FAR and FRR.

We also want to understand what the model is learning, not only how well it performs. For the FFT-based baseline, we can inspect which frequency components are most useful for separating different pocket placements and visualize them with bar plots or heatmaps. For the transformer model, we want to visualize the embedding space to see how well the model separates users, and, if possible, we may also use additional interpretability tools to inspect what parts of the input are driving its decisions.

Finally, we will evaluate the model on new data that we record from a few volunteers who were not part of the original dataset. They will carry the phone in different pockets while walking, and we will use this data to check how well the trained model generalizes to new people and to more realistic deployment conditions. This will give us a more practical sense of whether the system is robust outside the original training data. Given our deployment plan, this will initially mean sending data to our endpoint, but eventually it will mean live use on a simple app on a phone.

6 Model Usage

Our model could be used for:

- theft detection;
- suspicious activity detection;
- purchase confirmation based on recent motion behavior; and

- other lightweight security checks that require confirmation of identity.

Why: the system is relatively small, can run on-device, and has access to live motion data.

Who could use it: anyone with a smartphone, especially users who want an extra passive layer of security.

A user could download the app or access an API-based web app. If we can make the model small enough, we may be able to deploy it directly in the browser or as a lightweight local download. There would first be a short enrollment period where the user walks for a few minutes to create an identification embedding. After that, live data would be streamed in, interpolated and resampled to handle jitter, and then compared through cosine similarity or another comparison method against the known owner embedding.

Most of the time the system would remain idle. If it makes a prediction that suggests a possible threat, the app could trigger an alert, message, or notification on another device.

The final output is essentially a yes/no decision, so the user-facing result would likely be a notification on another device. To support that, we would need SMS, email, bot-based messaging, or a server that can receive the signal and send a ping to another device.

7 Risk Assessment

Some potential risks related to project development include data leakage across training and test sets, over-complexity of the transformer models for the task, and the need to learn new techniques for model compression and deployment.

To mitigate the risk of data leakage, as mentioned previous sections, we will split the data by subject and no sensor data will be split for test set. This means that the model will be trained on data from some subjects and tested on data from different subjects.

As transformer models are often large and computationally expensive deep learning algorithms, we may end up with overly-complicated model for the task that is lenient to overfitting. To mitigate this risk, we will start with a smaller transformer architecture and only increase the complexity if necessary. Moreover, the baseline model could be used as a reference point for comparison. We will also use regularization techniques such as dropout and early stopping to prevent overfitting.

One major risk is the use of live streaming data, since this is personal and sensitive information. For that reason, we would prefer to keep the system on-device rather than

stream sensor data to a server. In this way, we aim to minimize the risk of data breaches and privacy violations.

We may also face a trade-off between model size and accuracy, and we may need to learn deployment techniques that are specific to this use case. To prepare for that, we want to look early at projects such as the web `fake beer` drinking app to understand how live streaming sensor data is handled, and we may also need to explore mobile development.

The project is likely to be successful in terms of model accuracy, since the task has already been shown to be possible with high accuracy. What is less certain is how feasible the approach will be once compression and deployment constraints are added.

The main societal risk is misuse: The system could be used to detect, follow, or identify someone through their embedding, which would effectively leak biometric information. There are also user risks because this system would be an added security layer, and users might blame the model if it fails in a sensitive situation. In a real deployment, the model would need to achieve a very high level of reliability before it could be trusted.

8 Individual Learning Outcomes

Will: I would like to learn more about model compression as a general practice and about the nuance of deploying such a model. This connects to my interest in neuromorphic computing, because smaller neural networks are easier to simulate in that setting and can provide useful insight into how these ideas might be implemented in circuitry.

Haukur:

Tijje:

Efe:

9 Relation to Grading Specifications

In previous sections, we have already described how we will be pre-processing the data, conducting cross-validation, and optimizing hyperparameters.

For the evaluation of over/underfitting with regularization, we already mentioned that we will be using regularization techniques as well as early stopping to prevent overfitting.

In terms of novelty, we believe our project is novel as no previous work on the real world application of gait-based owner verification has been done. To this end, we aim

to compress the model to be able to run on a smartphone.

10 Timeline

The first week corresponds to the week of 20-26 April.

Task	**Week 1**	**Week 2**	**Week 3**	**Week 4**	**Week 5**	**Week 6**	**Week 7**	**Week 8**
Project Search/Selection	■							
Literature Search	■	■						
Proposal Writing	■	■						
Data Exploration		■	■					
Preprocessing			■	■				
Training baseline model			■	■				
Training main model				■	■			
Evaluation					■	■		
Model Deployment (API)					■	■		
Implementing Extras (Model Compression, Explainability & on-device deployment)				■	■	■		
Report Writing					■	■	■	■

Table 1: Proposed timeline for the project.

References

[1] MathWorks. Gyroscope block [digital image]. MathWorks Help Center, 2014. In *Gyroscope – Measure rotational speed around X, Y, and Z axes in rad/s*. Retrieved April 30, 2026, from `https://nl.mathworks.com/help/simulink/supportpkg/android_ref/gyroscope.html`.